

Lista de Exercícios - Classes abstratas, polimorfismo e interfaces

1. Explique por que não podemos ter construtores declarados com a palavra-chave *abstract*.
2. Defina uma classe para conter informações sobre um funcionário de uma empresa (classe *Funcionario*). Quais são os atributos dessa classe? Inclua entre eles o salário que o funcionário deve receber por hora trabalhada. Implemente, para essa classe, pelo menos dois métodos construtores: um que receba apenas o nome do funcionário e assuma valores padrão para os demais atributos (assuma que o funcionário deve receber dois reais por hora trabalhada); o segundo construtor deve receber, além do nome, o valor que o referido trabalhador deve receber por hora trabalhada. Identifique e implemente os demais métodos que achar conveniente para um objeto da classe *Funcionario*.
3. Crie a classe *FiguraGeometrica* que possui um método abstrato *descricao()*. Crie também as classes *Circulo*, *Quadrado* e *Triangulo* que são subclasses da classe *FiguraGeometrica* e implementam o método *descricao()* apropriado para sua classe. Por fim, crie uma classe *Principal* com um método *main* que cria um objeto de cada uma das classes e chama seus respectivos métodos *descricao()*.
 - O método *descricao()* deve exibir um texto que descreva a figura.
4. Crie uma classe *Desenho* que possui dois atributos do tipo *FiguraGeometrica* (criado na questão anterior) e suas respectivas coordenadas em um plano bidimensional. Escreva um construtor para a classe *Desenho* que inicialize todos os atributos através dos parâmetros. Implemente também o método *apresenta()* que, para cada *FiguraGeometrica* em um *Desenho*, informa suas coordenadas e imprime sua descrição. Por fim, crie uma classe executável, *Principal*, que cria dois objetos do tipo *Desenho* e chama seu método *apresenta*. O primeiro *Desenho* deve ser formado por um *Circulo* e um *Quadrado* e o segundo por um *Quadrado* e um *Triangulo*.
5. Crie uma interface *ItemDeBiblioteca* que declara quais campos e métodos uma classe que representa um item para empréstimo em uma biblioteca deve implementar. Essa interface é composta por um campo *maximoDeDiasParaEmprestimo* com valor 14 e os seguintes métodos:
 - *estaEmprestado* : retorna verdadeiro se o item estiver emprestado e falso caso contrário;
 - *empresta* : modifica para verdadeiro o estado de um campo que indica se o item está emprestado ou não;
 - *devolve* : modifica para falso o estado de um campo que indica se o item está emprestado ou não;

- *localizacao* : retorna um texto que informa o local do item na biblioteca (e.g: "corredor 2, prateleira D");
- *descricao* : retorna texto contendo uma descrição resumida do item (e.g.: "artigo da ECOP").

Implemente também a classe *Livro* que encapsula os dados genéricos sobre um livro e métodos para processar estes dados. Essa classe é composta pelos atributos *titulo*, *autor*, *numeroDePaginas* e *anoDaEdicao*, além dos seguintes métodos:

- construtor;
- *qualTitulo*: retorna o título do livro;
- *qualAutor*: retorna o autor do livro;
- *toString*: retorna os valores dos campos desta classe em formato textual.

Em seguida, escreva a classe *LivroDeBiblioteca* que herda os campos e métodos da classe *Livro* e implementa os métodos declarados na interface *ItemDeBiblioteca*. *LivroDeBiblioteca* também deve possuir um construtor e um método *toString*. Crie os atributos que forem necessários.

Por fim, crie a classe *DemoLivroDeBiblioteca* para demonstrar o uso de uma instância da classe *LivroDeBiblioteca*, isto é, criar um objeto do tipo *LivroDeBiblioteca* e executar seus métodos.

6. [FCC - 2023 - Copergás - PE - Analista / Suporte de Informática] A sobrecarga de métodos ocorre quando

- (A) um método de mesmo nome existe em classes diferentes em uma relação de herança.
- (B) há mais de um método com mesmo nome, recebendo parâmetros de tipos ou em quantidades diferentes, em uma classe.
- (C) um método recebe mais parâmetros do que pode suportar.
- (D) o mesmo método aparece na maioria das classes da aplicação.
- (E) um método recebe como parâmetro um valor que extrapola a capacidade do tipo usado para o recebimento.

7. [IDECAN - 2019 - IF-PB - Professor - Informática] Dadas as seguintes classes, todas no mesmo pacote:

```
public class Mamifero {
    protected void andar() {
        System.out.print("Mamifero andando");
        ouvir();
    }
    protected void ver() {
        System.out.print("Mamifero vendo");
    }
    protected void ouvir() {
```

```

        System.out.print("Mamifero ouvindo");
        ver();
    }
}

public class Primata extends Mamifero {
    protected void andar() {
        System.out.print("Primata andando");
        ouvir();
    }
}

public class Homem extends Primata {
    protected void ver() {
        System.out.print("Homem vendo");
    }
    public static void main(String[] args) {
        Mamifero m = new Homem();
        m.andar();
    }
}

```

Qual é a saída deste programa?

- (A) Mamífero andando Mamífero ouvindo Mamífero vendo
 - (B) Primata andando Mamífero ouvindo Homem vendo
 - (C) Uma exception será lançada: MethodNotFoundException
 - (D) Mamífero andando Mamífero ouvindo Homem vendo
 - (E) Primata andando Mamífero ouvindo Mamífero vendo
8. [PETROBRAS - 2012 - Analista de Sistemas Júnior - Engenharia de Software] Suponha que as classes Circulo, Desenho e Figura ocupem arquivos separados. Em qual código Java elas serão compiladas sem erros?

- (A)

```

package P1;
import P2.*;
public class Figura {
    protected double x,y;
    protected final double PI=0;
    Desenho d;
    abstract protected double dist(double x1,double y1);
}

package P1;
public class Circulo extends Figura {
    double r;
    public Circulo() {

```

```

        d.add(this);
        PI=3.14159;
    }
    public double raio() { return r; }
    public double centroX() { return x; }
    public double centroY() { return y; }
    protected double dist(double x1,double y1) {
        return Math.sqrt((x1-x)*(x1-x)+(y1-y)*(y1-y));
    }
}

```

```

package P2;
import java.util.List;
import P1.Figura;
public class Desenho {
    List<Figura> f;
    public void add(Figura p) { f.add(p); }
}

```

(B)

```

package P1;
import P2.*;

abstract public class Figura {
    protected double x,y;
    protected final double PI=0;
    Desenho d;
    abstract protected double dist(double x1,double y1);
}

```

```

package P1;
public class Circulo extends Figura {
    double r;
    public Circulo() {
        d.add(this);
        PI=3.14159;
    }
    public double raio() { return r; }
    public double centroX() { return x; }
    public double centroY() { return y; }
    private double dist(double x1,double y1) {
        return Math.sqrt((x1-x)*(x1-x)+(y1-y)*(y1-y));
    }
}

```

```

package P2;
import java.util.List;
import P1.Figura;

public class Desenho {

```

```
List<Figura> f;
public void add(Figura p) { f.add(p); }
}
```

(C)

```
package P1;
import P2.*;

abstract public class Figura {
    double x,y;
    final double PI=3.14159;
    Desenho d;
    abstract protected double dist(double x1,double y1);
}

package P1;

public class Circulo extends Figura {
    double r;
    public Circulo() { d.add(this); }
    public double raio() { return r; }
    public double centroX() { return x; }
    public double centroY() { return y; }
    public double dist(double x1,double y1) {
        return Math.sqrt((x1-x)*(x1-x)+(y1-y)*(y1-y));
    }
}

package P2;

import java.util.List;
import P1.Figura;
public class Desenho {
    List<Figura> f;
    public void add(Figura p) { f.add(p); }
}
```

(D)

```
package P1;
import P2.*;

public class Circ implements ICirculo {
    double cx, cy, r;
    public double raio() { return r; }
    public double centroX() { return cx; }
}

package P2;

public interface ICirculo {
```

```
double PI;
double raio();
double centroX();
double centroY();
}
```

(E)

```
package P1;
import P2.*;

public class Circ extends ICirculo {
    double cx, cy, r;
    public double raio() { return r; }
    public double centroX() { return cx; }
    public double centroY() { return cy; }
}

package P2;

public interface ICirculo {
    double PI=3.14159;
    double raio();
    double centroX();
    double centroY();
}
```

9. [PETROBRAS - 2011 - Analista de Sistemas Júnior - Engenharia de Software] Analise os fragmentos de código dados abaixo

```
package javaapplication1;
public interface Interface1 {
    public int metodoComum();
}

package javaapplication1;
public class Concreta1 implements Interface1 {
    public int metodoComum() {
        return(1);
    }

    public int metodoExotico() {
        return(2);
    }
}

package javaapplication1;
public class Main {

    public static void main(String[] args) {
```

```

        Interface1 x = new Concreta1();
        System.out.println(x.metodoComum());
        System.out.println(x.metodoExotico());
    }
}

```

O resultado, obtido ao tentar compilar e executar esse conjunto de classes, será

- (A) um erro de compilação, indicando que não é possível fazer uma conversão da classe Concreta1 para a classe Interface1.
 - (B) um erro de compilação, indicando que, no contexto de x, não existe metodoExotico.
 - (C) nenhuma saída e um erro em tempo de execução, indicando que, dada a conversão de Concreta1 para Interface1, não é possível acessar metodoExotico.
 - (D) impressão do número 1, seguida de um erro de tempo de execução, indicando que, dada a conversão de Concreta1 para Interface1, não é possível acessar metodoExotico.
 - (E) impressão dos números 1 e 2.
10. [UFC - 2013 - Analista de Tecnologia da Informação / Engenharia de Software] Na programação orientada a objetos, a possibilidade de haver mais de um método com o mesmo nome na mesma classe denomina-se:
- (a) Herança.
 - (b) Sobrescrita.
 - (c) Sobrecarga.
 - (d) Ligação tardia.
 - (e) Encapsulamento.
11. [UFSC - 2023 - Técnico de Tecnologia da Informação] Considere as seguintes definições relacionadas à programação orientada a objetos, com lacunas a preencher, e assinale a alternativa que preencha corretamente as três definições, considerando sua ordem.
1. _____ é a capacidade de objetos de classes distintas responderem a uma mesma chamada de método de maneiras diferentes. Isso permite que as subclasses redefinam o comportamento de métodos herdados da classe base.
 2. _____ é a capacidade de um objeto ter vários métodos com o mesmo identificador, mas com assinaturas de métodos diferentes. Isso permite que os objetos respondam a chamadas de métodos distintos, mas com identificadores idênticos, com base na quantidade e no tipo de argumentos fornecidos.
 3. _____ é a capacidade de uma subclasse substituir o comportamento de um método herdado da classe base. Isso permite que uma classe modifique o comportamento de um método para atender às suas próprias necessidades, mantendo a mesma assinatura de método.
- (A) Sobrecarga – Polimorfismo – Herança

- (B) Sobrescrita – Polimorfismo – Encapsulamento
(C) Polimorfismo – Sobrecarga – Herança
(D) Herança – Encapsulamento – Sobrescrita
(E) Polimorfismo – Sobrecarga – Sobrescrita
12. [FAB - CIAAR - 2023 - Oficial de Apoio da Aeronáutica - Análise de Sistemas] Existe uma gama de definições sobre a orientação a objetos. No sentido da relação das classes e dos acessos aos métodos, preencha as lacunas abaixo.
- Muitas classes podem ter acesso ____, porém, _____ esse método _____. A sequência de palavras que preenche corretamente as lacunas é:
- (A) a um mesmo método / cada classe executa / de maneira diferente
(B) a um mesmo método / outras classes executam / da mesma maneira
(C) somente a um método / outras classes executam / de maneira diferente
(D) somente a um método / cada classe executa / da mesma maneira
13. [FGV - PGM Niterói - 2023 - RJ • Analista de Tecnologia da Informação] Observe o método `liga()` do seguinte trecho de código escrito na linguagem Java.

```
class Aparelho {  
    public void liga() {}  
}  
  
class Geladeira extends Aparelho {  
    public void liga() {  
        System.out.println("Tomada");  
    }  
}  
  
class TV extends Aparelho {  
    public void liga() {  
        System.out.println("Controle remoto");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Aparelho geladeira = new Geladeira();  
        Aparelho tv = new TV();  
        geladeira.liga();  
        tv.liga();  
    }  
}
```

Em orientação a objeto, o uso de um método com comportamento diferente, como `liga()`, é realizado por meio do emprego de:

- (A) Lean;
- (B) Eventos;
- (C) Recursão;
- (D) Polimorfismo;
- (E) Encapsulamento.

14. [FAB - EEAR - 2023 - Sargento da Aeronáutica - Informática] Assinale a alternativa que completa a lacuna do texto abaixo.

A POO oferece um tipo especial de classe que não pode ser instanciada. Trata-se da classe _____

- (A) empacotada
- (B) concreta
- (C) abstrata
- (D) oculta

15. [Instituto Access - UFFS - 2023 - Técnico de Tecnologia da Informação] No que diz respeito à orientação a objetos, um princípio é definido como aquele em que as classes derivadas de uma única classe base são capazes de invocar os métodos que, embora apresentem a mesma assinatura, comportam-se de maneira diferente para cada uma das classes derivadas. É um mecanismo por meio do qual selecionam-se as funcionalidades utilizadas de forma dinâmica por um programa no decorrer de sua execução. Esse princípio é conhecido por

- (A) encapsulamento.
- (B) polimorfismo.
- (C) acoplamento.
- (D) herança.
- (E) coesão.

16. [UFPR - 2023 - IF-PR - Informática] O polimorfismo permite que uma mesma mensagem, tratada por um mesmo método, enviada para diferentes objetos, apresente resultados diferentes. Isso também permite que objetos com tipos diferentes sejam tratados da mesma forma. A hierarquia de classes Java apresentada utiliza o conceito de polimorfismo. Cada classe está salva em seu arquivo específico com o nome da classe e a extensão .java.

```
public interface Exame {  
    public abstract void mostrarPreparo();  
}  
  
public class ExameImagem implements Exame {  
    @Override  
    public void mostrarPreparo() {  
        System.out.println("EXAME DE IMAGEM PREPARO:");  
    }  
}
```

```

        System.out.println("Nenhum preparo necessario.");
    }
}

public class EcografiaTireoide extends ExameImagem {
}

public class ExameSangue implements Exame {
    @Override
    public void mostrarPreparo() {
        System.out.println("EXAMES DE SANGUE - PREPARO:");
    }
}

public class GlicemiaJejum extends ExameSangue {
    @Override
    public void mostrarPreparo() {
        System.out.println("GLICEMIA EM JEJUM - PREPARO:");
        System.out.println("Nao ingerir bebidas alcoolicas 72 horas antes do
        exame.");
        System.out.println("Jejum de 8 a 12 horas");
    }
}

import java.util.ArrayList;
import java.util.List;

public class AgendarExame {
    public static void main(String[] args) {
        List<Exame> examesPaciente = new ArrayList<>();
        examesPaciente.add(new GlicemiaJejum());
        examesPaciente.add(new EcografiaTireoide());
        for (Exame exame : examesPaciente) {
            exame.mostrarPreparo();
        }
    }
}

```

Qual o resultado da execução do código descrito no método main da classe AgendarExame?

- (A) O método não pode ser executado, pois a classe EcografiaTireoide não tem implementação para o método mostrarPreparo.
- (B) GLICEMIA EM JEJUM - PREPARO:
 Não ingerir bebidas alcoólicas 72 horas antes do exame.
 Jejum de 8 a 12 horas
 EXAME DE IMAGEM PREPARO:
 Nenhum preparo necessário.

- (C) EXAMES DE SANGUE - PREPARO:
GLICEMIA EM JEJUM - PREPARO:
Não ingerir bebidas alcoólicas 72 horas antes do exame.
Jejum de 8 a 12 horas EXAME DE IMAGEM PREPARO:
Nenhum preparo necessário.
- (D) GLICEMIA EM JEJUM - PREPARO:
Não ingerir bebidas alcoólicas 72 horas antes do exame.
Jejum de 8 a 12 horas
- (E) O método não pode ser executado, pois Exame não define um tipo para o objeto, já que não é uma classe e sim uma interface.

17. [BIO-RIO - 2016 - Prefeitura de São Gonçalo - RJ - Analista da Área Tecnológica] No que diz respeito à Orientação a Objetos, dois princípios são caracterizados a seguir.

I constitui um mecanismo que tem por objetivo organizar os dados que sejam relacionados, agrupando-os em objetos, reduzindo as colisões de nomes de variáveis e, da mesma forma, reunindo métodos relacionados às suas propriedades. Este padrão ajuda a manter um programa com centenas ou milhares de linhas de código mais legível e fácil de trabalhar e manter.

II constitui um mecanismo a partir do qual as classes derivadas de uma única classe base são capazes de invocar os métodos que, embora apresentem a mesma assinatura, comportam-se de maneira diferente para cada uma das classes derivadas. De acordo com este princípio, os mesmos atributos e objetos podem ser utilizados em objetos distintos, porém, com implementações lógicas diferentes.

Os princípios I e II são conhecidos respectivamente como

- (A) coesão e herança.
(B) polimorfismo e coesão.
(C) herança e acoplamento.
(D) acoplamento e encapsulamento.
(E) encapsulamento e polimorfismo.
18. [CESGRANRIO - 2024 - Banco da Amazônia - Técnico Científico - Tecnologia da Informação] Considere um sistema bancário em Java que possui a classe Cliente e suas subclasses, ClientePessoaFisica e ClientePessoaJuridica, onde Cliente é uma classe abstrata. Nesse sistema, um método getDesconto(valor) deve fornecer o cálculo do desconto para um tipo de cliente, de forma que os clientes do tipo pessoa física e os clientes do tipo pessoa jurídica tenham descontos diferenciados. Suponha que, utilizando corretamente os mecanismos associados à herança e ao polimorfismo, se deseje implementar essas classes de modo que o método getDesconto possa ser aplicado indistintamente a qualquer instância que tenha sido declarada como da classe Cliente.
- Para atender a essa condição, a implementação dessas classes deve possuir o método getDesconto

- (A) apenas na classe Cliente, identificando a qual subclasse o cliente pertence e calculando o desconto por meio de uma estrutura se-então dentro da implementação.
- (B) em todas as três classes, sendo possível, nesse caso, que a função getDesconto da classe Cliente seja abstrata.
- (C) apenas nas classes ClientePessoaFisica e ClientePessoaJuridica, pois não há instâncias da classe Cliente no sistema, já que é uma classe abstrata.
- (D) fora das três classes, dado que uma estrutura do tipo se-então deve ser usada para descobrir qual é a classe adequada.
- (E) implementada totalmente em todas as classes, sendo que a função getDesconto da classe Cliente chama a função getDesconto das classes ClientePessoaFisica e ClientePessoaJuridica de acordo com a necessidade.

19. [FGV - 2024 - DATAPREV - ATI - Arquitetura, Engenharia e Sustentação Tecnológica] Em um sistema de gerenciamento de pagamentos, existem as classes Pagamento (classe base), PagamentoCartao e PagamentoBoleto (ambas herdam de Pagamento). A classe Pagamento define o método realizarPagamento(), que é sobrescrito tanto por PagamentoCartao quanto por PagamentoBoleto para implementar comportamentos específicos de cada tipo de pagamento. Considere o seguinte código:

```
public void processarPagamento(Pagamento pagamento) {  
    pagamento.realizarPagamento();  
}
```

Assinale a opção que indica o conceito de orientação a objetos que está sendo aplicado quando o método realizarPagamento() é chamado em um objeto do tipo Pagamento, mas o comportamento específico é definido pelas subclasses (PagamentoCartao ou PagamentoBoleto).

- (A) Encapsulamento, pois o método realizarPagamento() está escondido da implementação interna.
 - (B) Herança, pois as classes filhas estão herdando o método realizarPagamento() da classe Pagamento.
 - (C) Sobrecarga de métodos, pois o método realizarPagamento() está definido com diferentes assinaturas.
 - (D) Polimorfismo, pois o método é definido na classe base, mas o comportamento é determinado pelas subclasses.
 - (E) Abstração, pois o método realizarPagamento() oculta a complexidade da lógica interna de pagamento.
20. [UFU-MG - 2023 - Analista de Tecnologia da Informação Área 1 - Desenvolvimento de Sites, Aplicações e Sistemas] Considere o caso de orientação a objeto, apresentado no código abaixo, para analisar as asserções apresentadas.

```
public class Phone{  
    public void callNumber(long number) {  
        System.out.println("Phone: Discando numero: " + number);  
    }  
}
```

```

    }

    public boolean isRinging(){
        System.out.println("Phone: Verificar se telefone esta tocando");
        boolean ringing = false;
        return ringing;
    }
}

public class SmartPhone extends Phone{
    public void sendEmail(String message, String address){
        System.out.println("SmartPhone: Enviando E-mail");
    }

    public String retrieveEmail(){
        System.out.println("SmartPhone: Recuperando e-mail");
        String messages = new String();
        return messages;
    }

    public boolean isRinging() {
        System.out.println("SmartPhone: Verificar se smartphone esta tocando");
        boolean ringing = false;
        return ringing;
    }
}

public class App {
    public static void main(String[] args) {
        new App();
    }
    public App() {
        SmartPhone smartPhone = new SmartPhone();
        testPhone(smartPhone);
        testSmartPhone(smartPhone);
    }

    private void testPhone(Phone phone){
        phone.callNumber(34999999999);
        phone.isRinging();
    }

    private void testSmartPhone(SmartPhone phone){
        phone.sendEmail("Oi", "teste@ufu.br");
        phone.retrieveEmail();
    }
}

```

FONTE: FINEGAN, Edward. OCA Java SE 8: Guia de estudos para o exame 1Z0-808.

Porto Alegre: Bookman, 2018.

- I O caso apresentado demonstra um exemplo simples de herança ao definir a classe “SmartPhone” com uma extensão da classe “Phone”; no entanto, há um erro no construtor App() quando é executada a linha testPhone(smartPhone), visto que o método testPhone() requer como argumento um objeto do tipo Phone.
- II Sabendo-se que o polimorfismo é unidirecional, o método testSmartPhone() não pode ser usado com um objeto Phone como seu argumento.
- III A execução da linha testPhone(smartPhone), descrita dentro do construtor App(), terá como resultado as respectivas mensagens: “Phone: Discando numero: 34999999999” e “SmartPhone: Verificar se smartphone está tocando”.
- IV A execução da linha testPhone(smartPhone), descrita dentro do construtor App(), terá como resultado as respectivas mensagens: “Phone: Discando numero: 34999999999” e “Phone: Verificar se telefone está tocando”.

Estão corretas apenas as asserções

- (A) II e IV.
- (B) II e III.
- (C) I e III
- (D) I e IV.

21. [IFRN - 2009 - Professor - Sistemas de Informação] O código, abaixo, apresenta a implementação de uma classe na linguagem de programação Java. Com base nessa classe, marque a alternativa verdadeira.

```
public final class Password {  
    private String senha;  
    public String getSenha() {  
        return senha;  
    }  
    public void setSenha(String senha) {  
        this.senha = senha;  
    }  
}
```

- (A) Esta classe não poderá ser estendida por outras classes, porém seus métodos poderão ser sobrescritos.
- (B) Esta classe não poderá ser estendida por outras classes.
- (C) Esta classe poderá ser estendida por outra classe.
- (D) Esta classe poderá ser estendida por outra classe, porém seus métodos não poderão ser sobrescritos.

22. [FGV - 2024 - Prefeitura de Caraguatatuba - SP - Técnico em Processamento de Dados] (adaptada) Em um jogo de estratégia online, você tem diferentes classes de personagens, como “Guerreiro” e “Mago”, que herdam de uma classe base chamada “Personagem”. A classe base possui um método chamado “ExecutarHabilidade”, que funciona de maneira diferente quando chamado por um personagem guerreiro em comparação com um personagem mago.

Considerando princípios de programação orientada a objetos, assinale a abordagem mais adequada para implementar essa diferenciação.

- (A) Utilizar variáveis de controle condicional dentro do método “ExecutarHabilidade” para verificar explicitamente o tipo da instância e ajustar o comportamento.
- (B) Criar versões específicas do método “ExecutarHabilidade” para “Guerreiro” e “Mago” com implementações distintas.
- (C) Marcar o método “ExecutarHabilidade” como abstrato na classe base “Personagem” e fornecendo implementações específicas nas classes derivadas.
- (D) Permitir que “Guerreiro” e “Mago” herdem métodos específicos de diferentes classes e personalizem o comportamento de “ExecutarHabilidade”.
- (E) Ocultar a implementação do método “ExecutarHabilidade” na classe base “Personagem” e redefinindo-o nas classes derivadas para comportamentos específicos.

Referências

- SANTOS, R. Introdução à programação orientada a objetos usando JAVA. 2. ed. Rio de Janeiro: Campus, 2013. 336p.
- DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017.
- BATISTA, Rogério da Silva; MORAES, Rafael Araújo de. Introdução à Programação Orientada a Objetos. 2013. Disponível em: <http://proedu.rnp.br/handle/123456789/611>. Acesso em: 18 ago. 2021.
- SANTANCHÈ, André. Classes: lista de exercícios. 2011. Disponível em: <https://www.ic.unicamp.br/~santanch/teaching/oop/exercicios/poo-exercicios-02-classes-v01.pdf>. Acesso em: 30 mar. 2022.
- BACALÁ JÚNIOR, Sílvio. Revisão de POO em Java: lista de exercícios 1. 2022. Disponível em: <http://www.facom.ufu.br/~bacala/POO/lista1.pdf>. Acesso em: 30 mar. 2022.
- INSTITUTO METRÓPOLE DIGITAL. Programação Orientada a Objetos. 2015. Disponível em: <https://materialpublic.imd.ufrn.br/curso/disciplina/5/8>. Acesso em: 21 out. 2020. 2ª Edição. ISBN: 978-85-7064-002-4.
- GARCIA, Islene Calciolari. MC202 - Estruturas de Dados: Lista de Exercícios 1. Disponível em: <https://www.ic.unicamp.br/~islene/mc202/lista1.pdf>. Acesso em: 24 maio 2022.

- BACALÁ JÚNIOR, Sílvio. Exceções. Disponível em: <https://www.facom.ufu.br/~bacala/POO/08-CriandoExcecoes.pdf>. Acesso em: 31 maio 2022.
- FERREIRA, Nickerson. Lista de Exercícios - Herança. Disponível em: <https://docente.ifrn.edu.br/nickersonferreira/disciplinas/programacao-estruturada-e-orientada-a-objetos/lista-de-exercicios-heranca/>. Acesso em: 8 mar. 2023.
- USP. Parênteses Balanceados. Disponível em: https://panda.ime.usp.br/panda/static/pythonds_pt/03-EDBasicos/06-ParentesesBalanceados.html. Acesso em: 23 mar. 2024.
- GOVERNO DO ESTADO DE RONDÔNIA. Edital n. 287-2022-SEGEp-GCP-Abertura Concurso Publico Secretaria de Estado da Assistencia e do Desenvolvimento Social-SEAS-RO-Atualizado-Retificacao I. 2022. Disponível em: <https://rondonia.ro.gov.br/publicacao/16-11-2022-edital-n-287-2022-segep-gcp-abertura-concurso-publico-secretaria-de-estado-da-assistencia-e-do-desenvolvimento-social-seas-ro/>. Acesso em 20 ago. 2024.
- GOVERNO DO ESTADO DE RORAIMA. Edital n. 001/2022 - Secretaria de Estado da Fazenda de Roraima (SEFAZ/RR). 2022. Disponível em: <https://concurso.idecan.org.br/Concurso.aspx?ID=60>. Acesso em: 22 ago. 2024.
- UNIVERSIDADE FEDERAL DE UBERLÂNDIA. Edital 27/2023 - Pró-Reitoria de Gestão de Pessoas - PROGEp. 2023. Disponível em: <https://www.portalselecao.ufu.br/servicos/Edital/cronograma/1334>. Acesso em: 27 ago. 2024.
- GOVERNO DO ESTADO DE MATO GROSSO - Edital n. 01/2023 - Universidade do Estado de Mato Grosso - UNEMAT. 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/78029/unemat_2023_ptes_edital_n_1-edital.pdf. Acesso em: 31 ago. 2024.
- TRIBUNAL REGIONAL DO TRABALHO DA 14ª REGIÃO - RO E AC. Edital n. 01/2022 - Fundação Carlos Chagas. 2022. Disponível em: https://www.concursosfcc.com.br/concursos/trt14122/edital_de_abertura_trt14122_26_09_22.pdf. Acesso em: 31 ago. 2024.
- PREFEITURA MUNICIPAL DE RIO BRANCO - AC. Edital complementar n. 01/2024 - Instituto Verbena - Universidade Federal de Goiás. 2024. Disponível em: https://sistemas.institutoverbena.ufg.br/2024/concurso-rio-branco-ed1/sistema/arquivos/editais/EDITAL_COMPLEMENTAR_N%C2%BA.1.pdf. Acesso em: 30 ago. 2024.
- FUNDAÇÃO GETÚLIO VARGAS. Edital n. 1/2024 - Comissão de Valores Mobiliários - CVM. 2024. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77433/cvm_2024_edital_n_1-edital.pdf. Acesso em: 30 ago 2024.
- INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA - Edital n. 148/2018 - Instituto de Desenvolvimento Educacional, Cultural e Assistencial Nacional. 2012. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/18598/if_pb_2018_edital_n_148-edital.pdf. Acesso em: 31 ago. 2024.

- COMPANHIA PERNAMBUCANA DE GÁS - COPERGÁS - Edital n. 01/2022. - Fundação Carlos Chagas. 2022. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/73666/copergas_pe_2022_edital_n_1-edital.pdf. Acesso em: 31 ago. 2024.
- INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS - Edital n. 01/2023 - Fundação Getúlio Vargas. 2023. Disponível em: https://conhecimento.fgv.br/sites/default/files/concursos/edital-retificado-_tecnologista.pdf. Acesso em: 31 ago. 2024.
- IFRN. Concurso público para Professor de Sistemas de Informação - Edital Nº. 04/2009-DIGPE.2009. Disponível em: <https://www.pciconcursos.com.br/provas/download/professor-sistemas-de-informacao-if-rn-if-rn-2009>. Acesso em: 09 set. 2024.
- PETROBRAS - Edital n. 5, PSP-RH-1/2012 - Processo Seletivo Público para Preenchimento de Vagas e Formação de Cadastro em Cargos de Nível Superior e de Nível Médio - 2012. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/1884/petrobras_2012_diversos_cargos-edital.pdf. Acesso em: 11 set. 2024.
- PETROBRAS - Edital n.4, PSP-RH-1/2011 - Processo Seletivo Público para Preenchimento de Vagas e Formação de Cadastro Reserva em Cargos de Nível Superior e de Nível Médio. 2011. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/1346/petrobras_2011_edital_psp_rh_1_2011-edital.pdf. Acesso em: 25 set. 2024.
- UFC - Edital n.262/2013 - Concurso Público para Provimento de Cargos Técnico-Administrativos em Educação. 2013. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/17365/ufc_2013_edital_n_262-edital.pdf. Acesso em: 29 set. 2024.
- UFSC - Edital n. 002/2023/DDP - do Concurso Público para provimento de cargos da carreira Técnico-Administrativa em Educação (TAE). 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77063/ufsc_2023_edital_n_2-edital.pdf. Acesso em: 16 out. 2024.
- UFRJ - Edital n. 843/2023 - Concurso Público para provimento de vagas de cargos Técnico-Administrativos. 2023. Disponível em https://arquivos.qconcursos.com/regulamento/arquivo/77533/ufrj_2023_tecnico_administrativo_em_educacao_nivel_superior_edital_n_490-edital.pdf. Acesso em: 16 out. 2024.
- CENTRO DE INSTRUÇÃO E ADAPTAÇÃO DA AERONÁUTICA - Concurso Público para Oficial de Apoio da Aeronáutica - Análise de Sistemas. 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77188/ciaar_2024_oficial_de_apoio_eaoap-edital.pdf. Acesso em: 13 nov. 2024.
- PROCURADORIA GERAL DO MUNICÍPIO DE NITERÓI - Edital n. 01/2023 - Concurso público para o provimento de vagas nos cargos de nível superior de analista processual, Analista contábil e analista de tecnologia da informação e para nível médio de técnico de Procuradoria da procuradoria geral do município de Niterói - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/69167/pgm_niteroi_2023_edital_n_1-edital.pdf. Acesso em: 13 nov. 2024.

- ESCOLA DE ESPECIALISTAS DE AERONÁUTICA - EEAR - 2024 - Sargento - EAGS - Exame de Admissão ao Estágio de Adaptação à Graduação de Sargento da Aeronáutica de nível médio/técnico. 2024. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77026/ear_2024_sargento_eags-edital.pdf. Acesso em: 13 nov. 2024.
- UNIVERSIDADE FEDERAL DA FRONTEIRA SUL - UFFS - Edital n. 59/GR/UFFS/2023. Concurso público para provimento de cargos efetivos de técnico administrativos de nível médio e superior - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77092/uffs_2023_edital_n_59-edital.pdf. Acesso em: 27 nov. 2024.
- ESCOLA DE SAÚDE E FORMAÇÃO COMPLEMENTAR DO EXÉRCITO - EsFCEEx. Concurso público para quadro complementar e capelães - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77515/esfcex_2024_quadro_complementar_e_capelaes-edital.pdf. Acesso em: 27 nov. 2024.
- UNIVERSIDADE FEDERAL DO RIO DE JANEIRO - UFRJ - Edital nº 491, de 29 de abril de 2023. Concurso Público para provimento de vagas de cargos Técnico-Administrativos - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77534/ufrj_2023_tecnico_administrativo_em_educacao_nivel_medio_tecnico_edital_n_491-edital.pdf. Acesso em: 27 nov. 2024.
- UNIVERSIDADE FEDERAL DO ESPÍRITO SANTOS - UFES - Edital n. 46/2023 - Concurso público para os cargos técnico-administrativos em educação - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77660/ufes_2023_edital_n_42-edital.pdf. Acesso em: 27 nov. 2024.
- ASSEMBLEIA LEGISLATIVA DO ESTADO DO MARANHÃO - ALEMA - Edital n. 01/2023. Concurso público para provimento de vagas e cadastro de reserva Do quadro de pessoa - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/76911/al_ma_2023_edital_n_1-edital.pdf. Acesso em 28 nov. 2024.
- UNIVERSIDADE FEDERAL DO MARANHÃO - UFMA - Edital n. 172/2023-PROGEP. Concurso público para pessoal técnico-administrativo em educação - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77323/ufma_2023_edital_n_120-edital.pdf. Acesso em 28 nov. 2024.
- TRIBUNAL REGIONAL FEDERAL DA 2ª REGIÃO - Edital n. 01/2016. Concurso Público de Provas, destinado à formação de cadastro reserva para provimento de cargos do Quadro de Pessoal do Tribunal Regional Federal da 2ª Região e da Justiça Federal de Primeiro Grau das Seções Judiciárias do Rio de Janeiro e do Espírito Santo - 2016. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/11724/trf_2_regiao_2016-edital.pdf. Acesso em 20 dez. 2024.
- CÂMARA MUNICIPAL DE ANÁPOLIS-GO - Edital n. 01/2023. Concurso Público para provimento dos cargos efetivos da Câmara Municipal de Anápolis – GO - 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/78492/camara_de_anapolis_go_2023_edital_n_1-edital.pdf. Acesso em 23 dez. 2024.

- INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PARANÁ - Edital n. 160/2022. Concurso Público para provimento de cargos da Carreira de MAGISTÉRIO DO ENSINO BÁSICO, TÉCNICO E TECNOLÓGICO - 2022. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/76995/if_pr_2022_edital_n_160_professor-edital.pdf. Acesso em 26 dez. 2024.
- INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA GOIANO (IF GOIANO) - Edital n. 19/2023. Concurso Público para provimento de cargos do quadro de pessoal Técnico-Administrativo em Educação (TAE). 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77428/if_goiano_2023_tecnico_administrativem_educacao_edital_n_19-edital.pdf. Acesso em 30 dez. 2024.
- BANCO DA AMAZÔNIA S.A. - Edital n. 02/2024. Concurso público para preenchimento de vagas e formação de cadastro em cargos de Níveis médio e superior - Técnico científico. 2024. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/80493/banco_da_amazonia_2024_tecnico_edital_n_1-edital.pdf. Acesso em 02 jan. 2025.
- Prefeitura de Caraguatatuba - SP. Edital n. 03/2023. Concurso Público para provimento de vagas para diversos cargos para a Prefeitura de Caraguatatuba - Técnico em Processamento de Dados. 2023. Disponível em https://arquivos.qconcursos.com/regulamento/arquivo/78976/prefeitura_de_caraguatatuba_sp_2023_edital_n_3-edital.pdf. Acesso em 02 jan. 2025.
- Prefeitura de Rio Branco - AC. Edital n. 01/2024. Concurso Público para provimento dos cargos efetivos do quadro de pessoal do Município de Rio Branco - AC. 2024. Disponível em https://arquivos.qconcursos.com/regulamento/arquivo/79315/prefeitura_de_riobranco_ac_2024_edital_n_1-edital.pdf. Acesso em 02 jan. 2025
- DEFENSORIA PÚBLICA DO ESTADO DA PARAÍBA - Edital n. 0001/2021. Concurso Público para Analista de Desenvolvimento de Sistemas. 2021. Disponível em https://arquivos.qconcursos.com/regulamento/arquivo/40263/dpe_pb_2021_edital_n_001-edital.pdf. Acesso em 02 jan. 2025.
- PREFEITURA MUNICIPAL DE SÃO GONÇALO (RJ) - Edital n. 02/2016. Concurso Público para Analista na Área Tecnológica. 2016. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/8108/prefeitura_de_sao_goncalo_rj_2016_edital_n_02-edital.pdf. Acesso em 06 jan. 2025.
- UNIVERSIDADE FEDERAL DE PERNAMBUCO - Edital n. 10/2023. Concurso Público para provimento de cargos técnico-administrativos. 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77851/ufpe_2023_tecnico_administrativoem_educacao_edital_n_10-edital.pdf. Acesso em 06 jan. 2025.
- PREFEITURA MUNICIPAL DE JARU (RO) - Edital n. 001/2023. Concurso Público para provimento de cargos e cadastro reserva para seu quadro de pessoal, 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/79033/prefeitura_de_jaru_ro_2023_edital_n_1-edital.pdf. Acesso em 07 jan. 2025.

- UNIVERSIDADE FEDERAL DO PERNAMBUCO (UFPE) - Edital n. 10/2023. Concurso Público para provimento de cargos técnico-administrativos. 2023. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/77851/ufpe_2023_tecnico_administrativo_em_educacao_edital_n_10-edital.pdf. Acesso em 07 jan. 2025.
- EMPRESA BRASILEIRA DE CORREIOS E TELÉGRAFOS - Edital n. 271/2024. Concurso Público, para o provimento de vagas, sob o regime celetista, em cargos, do quadro de pessoal da Empresa Brasileira de Correios e Telégrafos. 2024. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/79364/correios_2024_analista_edital_n_271-edital.pdf. Acesso em 07 jan. 2025.
- EMPRESA DE TECNOLOGIA E INFORMAÇÕES DA PREVIDÊNCIA – DATAPREV S/A - Edital n. 1/2024. Concurso público para provimento de vagas e formação de cadastro de reserva para cargos do quadro de pessoal da Empresa de Tecnologia e Informações da Previdência. 2024. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/79983/dataprev_2024_edital_n_1-edital.pdf. Acesso em 14 jan. 2025.
- COMPANHIA DE ÁGUA E ESGOTOS DA PARAÍBA (CAGEPA) - Edital n. 1/2024. Concurso público para o provimento de vagas em cargos de nível superior e de nível médio técnico. 2024. Disponível em: https://arquivos.qconcursos.com/regulamento/arquivo/78089/cagepa_pb_2024_edital_n_1-edital.pdf. Acesso em 14 jan. 2025.